

Training Artificial Neural Networks on Unbalanced Data

Joshua Frankie Rayo
2011-19612
CS 280 Programming Assignment 2
October 18, 2016

Contents

1	Introduction	1
2	Methodology	1
2.1	Dealing with Unbalanced Data	2
2.2	Dataset Partitioning	3
2.3	Network Learning	3
3	Discussion of Results	4
4	Conclusion	11

1 Introduction

This work deals with using artificial neural networks as an intelligent system for classifying objects that is trained under highly unbalanced dataset. An approach to train the network is to generate data from what is already given. To upsample the dataset, SMOTE (Synthetic Minority Over-sampling Technique) algorithm has been used. Training parameters are tweaked to make good results in training and validation and to check the respective errors if they are as low as possible. The parameters include number of nodes in each layer, learning rate, and error tolerance criterion. The program is implemented using Python and NumPy package to enable code vectorization and make computations faster.

2 Methodology

The neural network is made using Python which is a high level, general-purpose programming language relatively easy to learn and has very good features in

list manipulation and vectorized computations. Also, the syntax for matrix computations of NumPy package is quite similar to MATLAB[6].

2.1 Dealing with Unbalanced Data

In the given dataset there are classes that have more instances than others. What does it mean to train an unbalanced data? Training the neural network with this kind of data might lead to overfitting and tend to assign objects in the dominant class most of the time[1]. This might lead to high accuracy but having a model with high accuracy is not really effective in prediction, which is known as the *accuracy paradox*[2].

There are several approaches in dealing with unbalanced data[4] (but is not limited to the following):

- Gather more data as much as possible
- Resample the dataset, meaning to generate new data
- Downsample the dataset, meaning to remove some instances
- Upsample the dataset, meaning to generate synthetic samples from the existing data
- Use a cost-sensitive classifier to minimize misclassification of the different classes

One of these approaches can be adapted depending on the given dataset and other restrictions. Gathering more data or resampling is impossible because its source is of unknown nature. There is nothing that can be inferred physically given only a sequence of numbers. Moreover, using a cost-sensitive classifier is not favorable since we do not know what penalty costs will be assigned for each misclassification. Downsampling the dataset is risky since there might be very important instances that might be removed. Therefore upsampling the dataset is the most feasible of the approaches given above. The SMOTE (Synthetic Minority Over-sampling Technique) algorithm is an upsampling approach which make data distribution more balanced by generating more instances from the minority classes as outlined by the algorithm below[3].

1. Take the difference between the feature vector under consideration and its nearest neighbor that belongs on the same class.
2. Multiply this difference by a random number between 0 and 1 and add it to the feature vector under the same class.

Mathematically, the synthetic instance is represented as $x_{new} = x_1 + \text{rand}(0, 1) * x_2$ where x_1, x_2 are neighboring features that belong to the same class. There are many ways to define a 'neighbor', perhaps through a distance function which can be Euclidean or L_1 -norm distance. This causes the selection of a random

point along the line segment between two specific features. In effect, it forces the decision region of the minority class to become more general[3].

In this particular work, synthetic instances are generated from the minority classes to make all class instances equal in population. The distance function used in determining nearby vectors is the L_1 -norm condition defined as $\sum_{i=0}^n \|x(i) - y(i)\| < 0.001$.

The table below show statistics regarding the dataset given and the number of generated instances per class:

label	count	generated	total
1	1625	0	1625
2	233	1392	1625
3	30	1595	1625
4	483	1142	1625
5	287	1338	1625
6	310	1315	1625
7	52	1573	1625
8	466	1159	1625
total	3486	9514	13000

2.2 Dataset Partitioning

After using SMOTE algorithm, the total number of instances is now 13,000 with each of the classes equally represented. The new dataset is divided into two partitions A and B. Partition A consists of 6,500 instances that includes original and synthetic data while partition B also have 6,500 instances that are all synthetic. The former is used for training and the latter is used for validating the network. In this manner the original data is still be used for training the network and original information are taken into account.

2.3 Network Learning

There are several parameters to be considered in training the network:

- Number of input neurons
- Number of hidden layers
- Number of neurons per layer
- Number of output neurons
- Learning rate η
- Error tolerance $\hat{J}$

The number of input neurons is set to 354, which is the number of features in an instance, while the number of output neurons is 3 which can represent 8

classes in binary form. The number of hidden layers are fixed into two. These values are specific to this particular problem. The learning rate is set to 1.0 which is the rate of gradient descent. Larger learning rate is being more aggressive in finding a minimum.

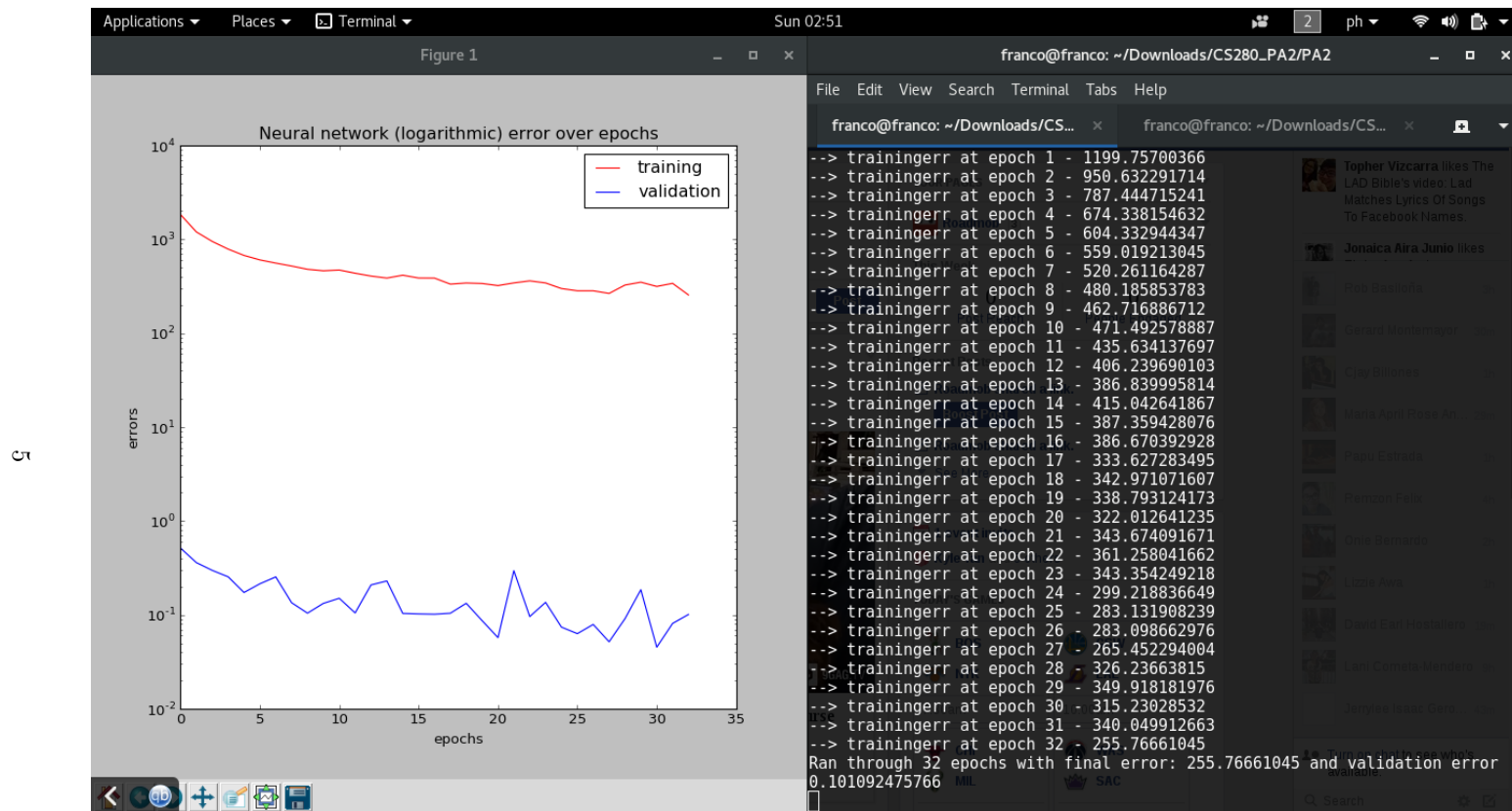
In setting the number of nodes in the hidden layers, it is suggested from [5] that the number of synaptic weights should be large enough to learn similarities of the instances that belong to same class and small enough to learn the differences of these similar instances. Moreover, the neurons of the first layer form the hyperplanes that separates the points, those from the second layer form the regions and the output layer form the classes[5]. Since there are only 8 classes, having 8 neurons in the first (8 separating hyperplanes) and second layers (8 sets of regions) is a good choice. However, both has been increased to 16 hoping that the network might capture the bigger picture as much as possible. In simulating the network, there is no significant gain in accuracy using only 8 nodes or 16 nodes at both layers.

The error in each instance i is defined as the sum of squared errors in output $\epsilon(i) = \frac{1}{2} \|\mathbf{y} - \hat{\mathbf{y}}\|^2$, while the error in an epoch is the sum of errors from m instances $J = \sum_{i=0}^m \epsilon(i)$. The sample code needs to be modified since it only checks for the last error in an epoch and not the sum.

In training the network, random values are assigned for weights and biases generated from the uniform distribution centered at -0.5. For each epoch, instances are drawn at random without replacement to make sure the network will not deduce any pattern that can produce bias during prediction. The labels (1-8) are of course converted to its binary representation (000-111) to be accepted by the network. For producing output, each neuron will be assigned 1 if its value is greater than 0.5 and 0 otherwise. Note that this is binary again and shall be converted back to its decimal representation for the final class labeling.

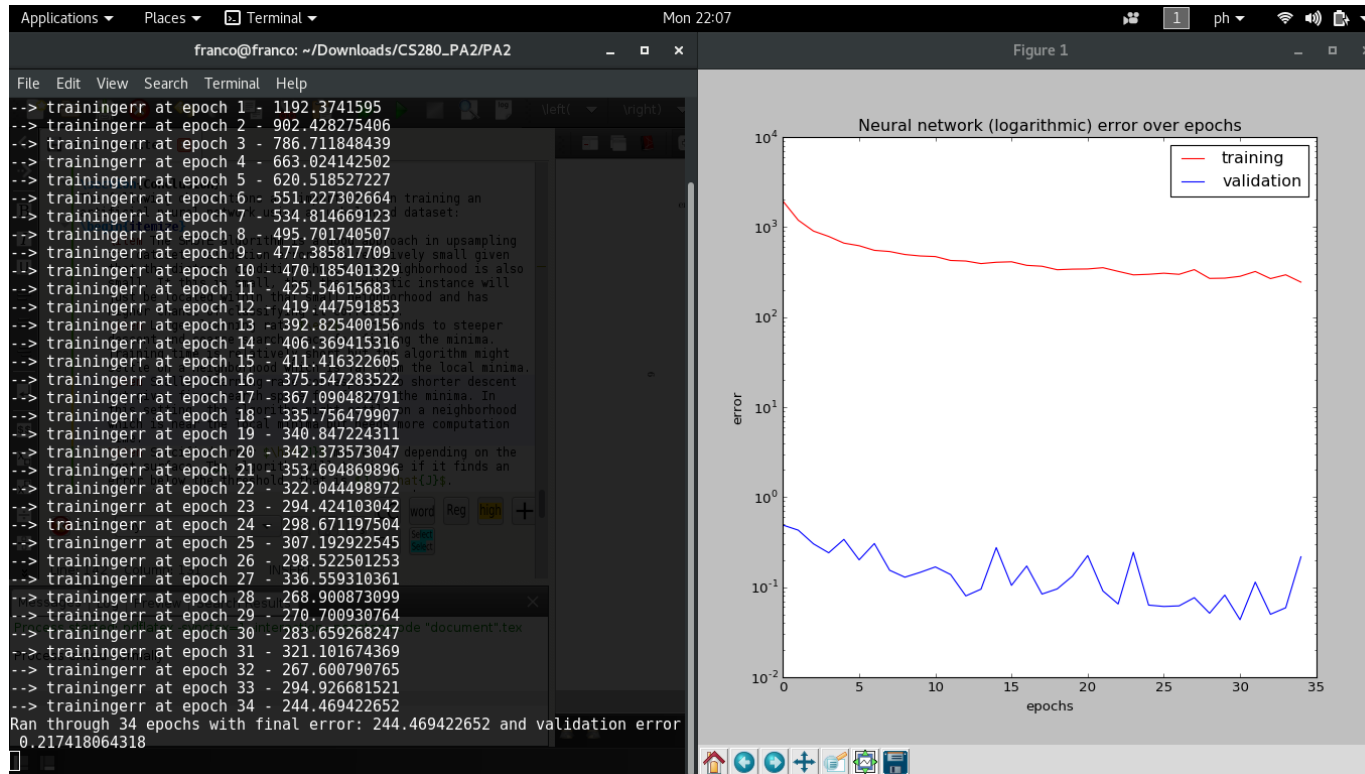
3 Discussion of Results

The network is ran using a laptop with Kali Linux OS, 8Gb RAM and Intel Core i3 @ 2.10GHz x 4 processors. For each epoch, training error for partition A and validation error for partition B is computed. The latter is computed as the percentage of misclassification in the validation set. In training the network, it is observed that the errors are large since each instance has 354 features and the cost function is quadratic. The figure below shows learning with $\hat{J} = 300$ and $\eta = 1.5$.



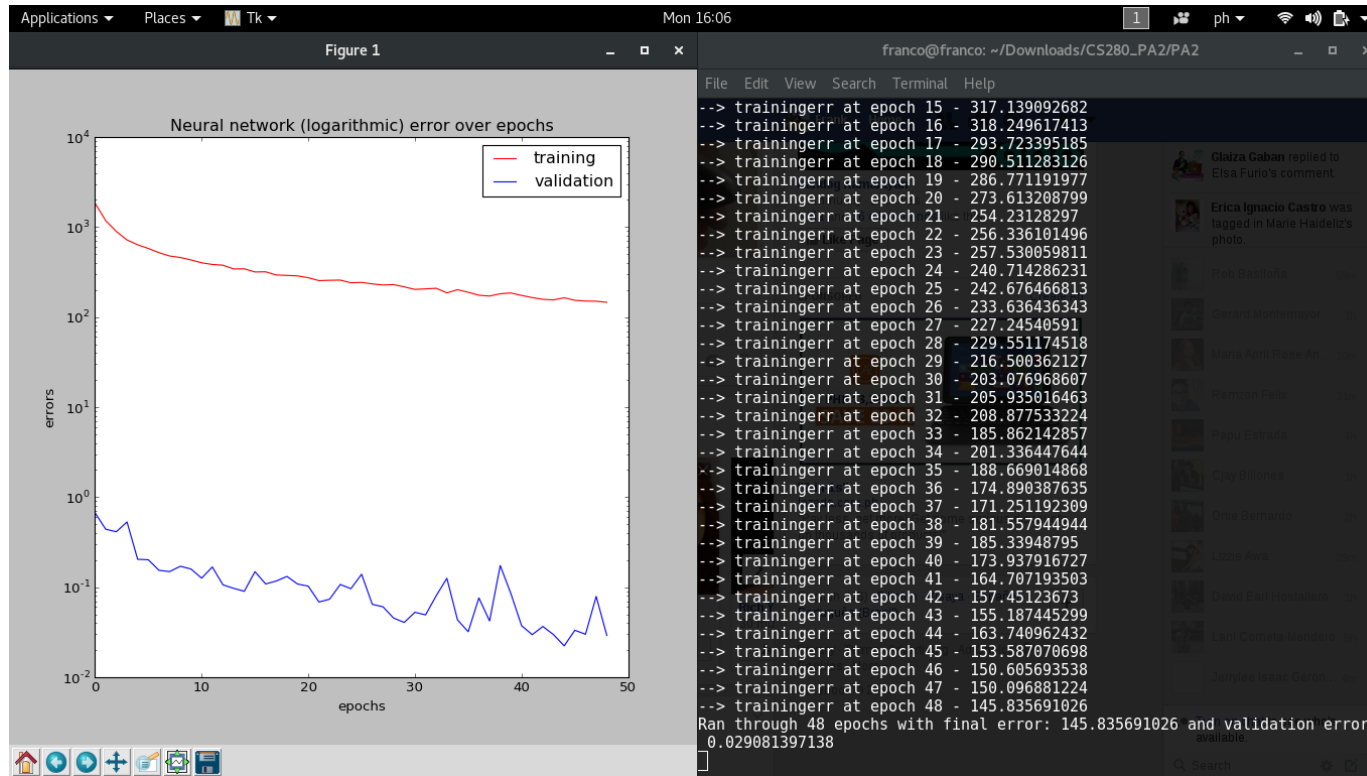
At first, training error quickly decrease and began oscillating at $J \approx 200 - 300$ in searching for the minimum. In this situation the local minimum is within that region and the gradient descent algorithm searches for a better estimate relative to

error tolerance. The next figure below shows a simulation where $\hat{J} = 250$ and $\eta = 1.5$



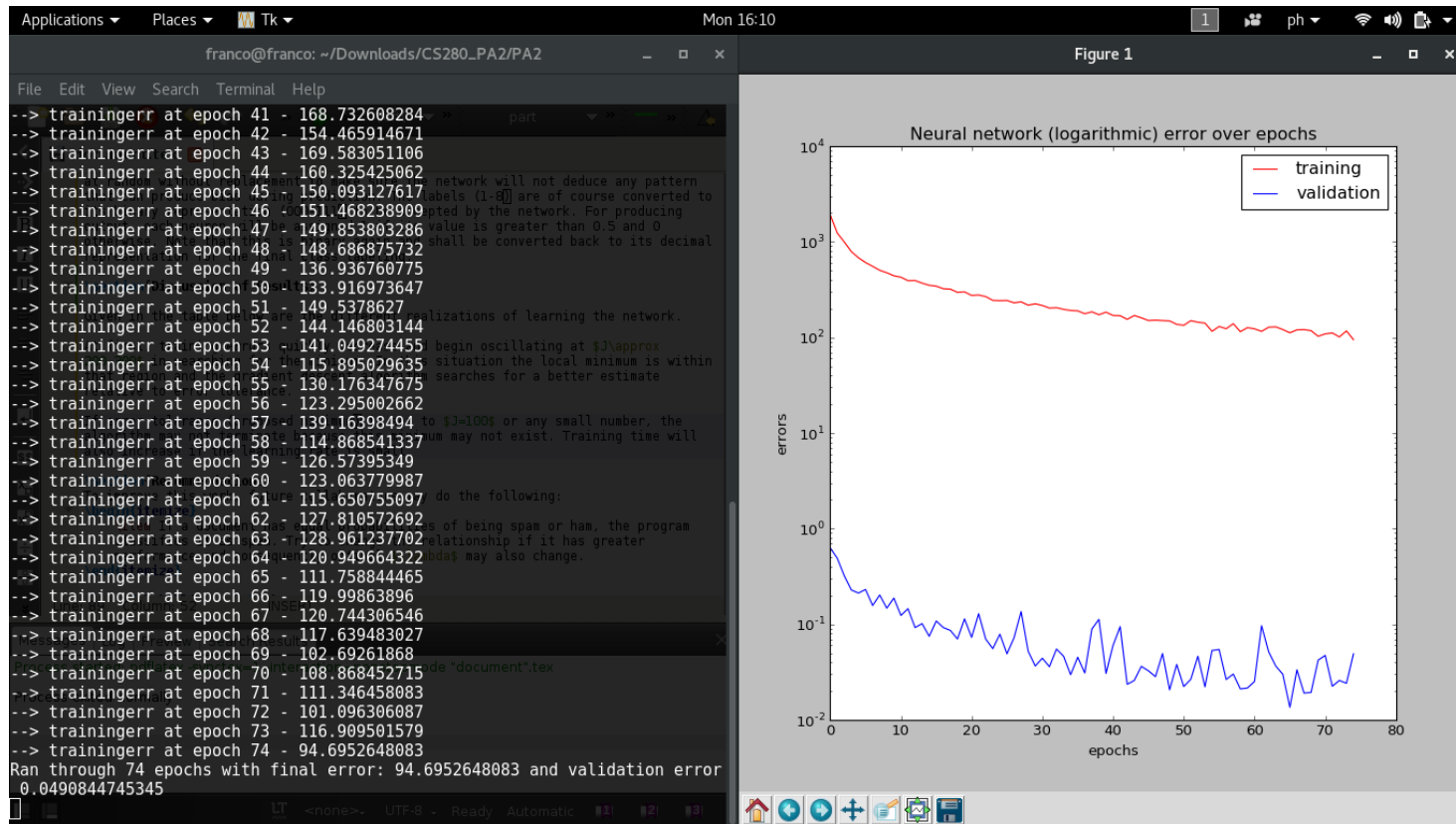
Training time increases if the learning rate is small and the algorithm is not much aggressive. If error tolerance (proposed

minimum) is set to $\hat{J} = 100$ or sufficiently small number, the algorithm may not terminate because this minimum might not exist depending on the cost surface. Small learning rate means finer search space in finding the minimum thus more time is required. The figure below shows a simulation where $\hat{J} = 150$, $\eta = 0.5$:

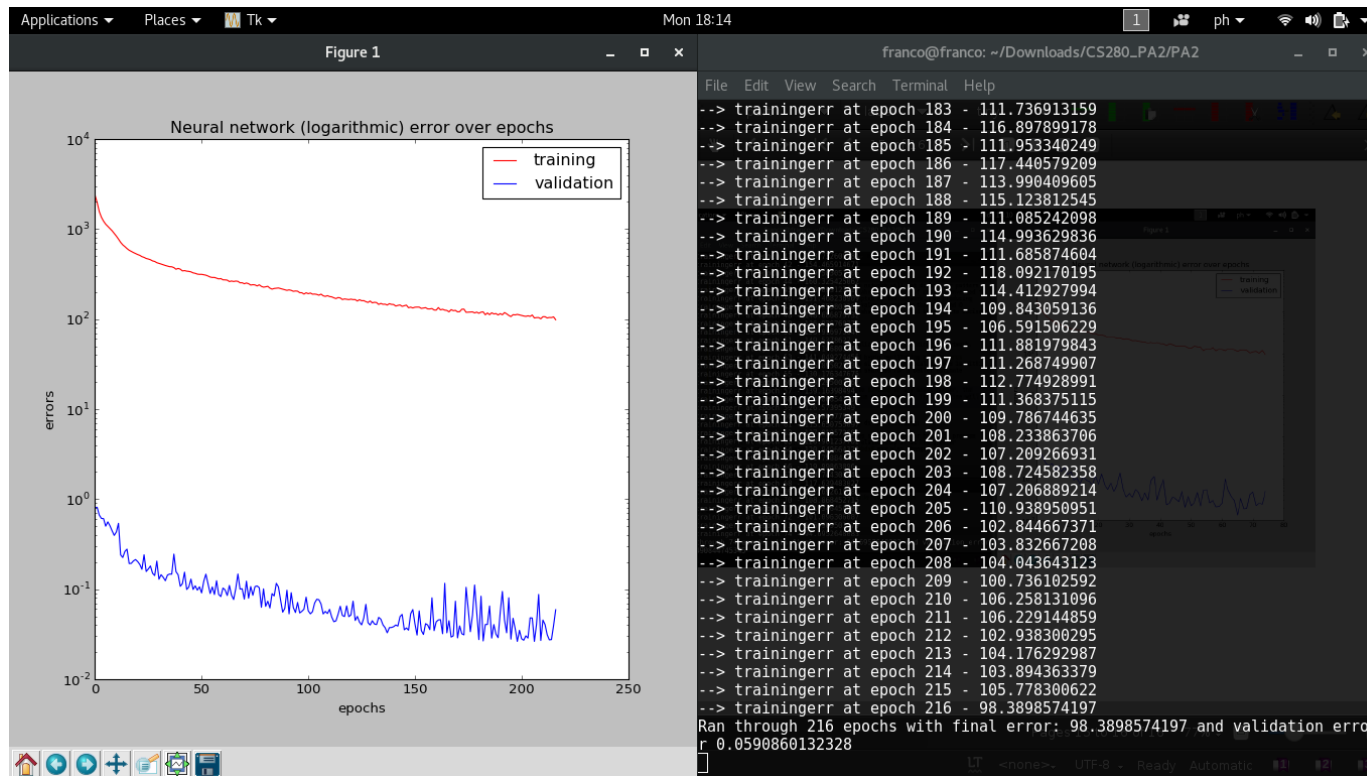


In this realization where $\hat{J} = 100$ and $\eta = 0.5$, learning takes 74 epochs with training error of 94.6953 and validation error of 0.0491.

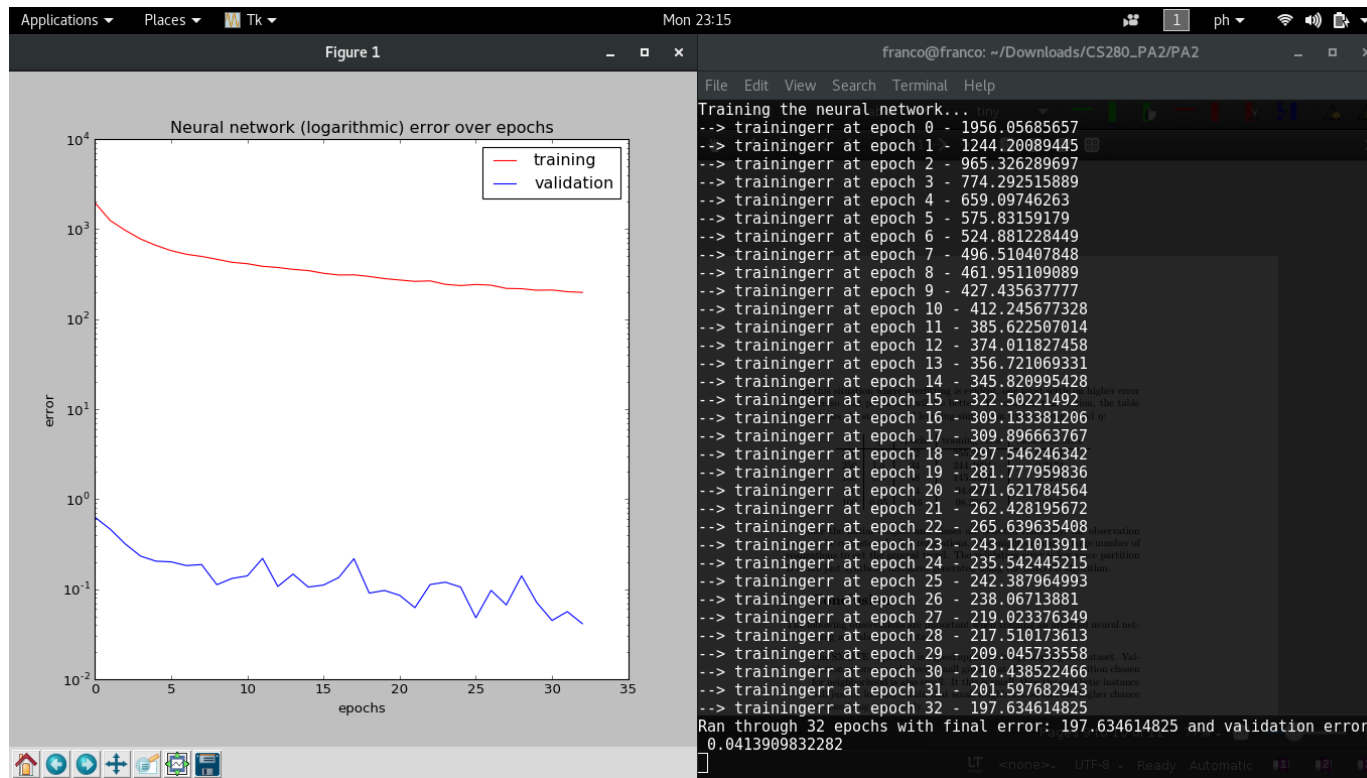
∞



This might ring bells: small validation error means great accuracy. But as you can see the graph, the validation error decreases at first then reaches minimum but tends to grow again with rapid oscillations. This is a sign of overfitting the dataset. Another instance that leads to this situation is having $\hat{J} = 100$ and $\eta = 0.05$ as given in the figure below.



In this situation where overfitting is evident, one must settle on higher error tolerance so that prediction will be better. Lastly, the figure below shows a simulation with recommended parameters $\hat{J} = 200$ and $\eta = 0.5$. This shows an error of 197.6346 and validation error of 0.0413 in 32 epochs.



To conclude this section, the table below shows the summary of learning simulation with varying $\hat{J}$ and η :

$\hat{J}$	η	epochs	training error	validation error
300	1.5	32	255.7666	0.1011
250	1.5	34	244.4694	0.2174
200	0.5	32	197.6346	0.0413
150	0.5	48	145.8357	0.0291
100	0.5	74	94.6953	0.0491
100	0.05	216	98.3899	0.0591

Since the initial weights and biases are selected randomly, the observations may not be consistent for few realizations. One might simulate a large number of realizations to get the general trend. The validation error is low since partition B is just synthetic instances generated using the SMOTE algorithm. At this point, unbalanced data has been addressed and finally the neural network is now ready for prediction.

4 Conclusion

The following observations are important when training an artificial neural network using an unbalanced dataset:

- The SMOTE algorithm is a good approach in upsampling the dataset. Validation errors are relatively small given that the distance condition chosen for neighborhood is also small. The synthetic instance will just be located within that small neighborhood and has higher chance of classifying it correctly.
- Larger learning rate η corresponds to steeper descent and coarse search space for finding the minima. Training time is relatively short but the algorithm might settle on a neighborhood which is far from the local minima.
- Smaller learning rate corresponds to shorter descent but gives finer search space for finding the minima. In this situation, the algorithm might settle on a neighborhood which is near the local minima. More computation time is needed to get better results.
- Specified error $\hat{J}$ may exist depending on the cost surface. The algorithm will terminate if it finds an error below the threshold, that is $J < \hat{J}$. Otherwise, learning process may not terminate.
- The number of nodes in the hidden layer determines the separation of points and classes of the unions of regions in the corresponding hyperspace. Increasing the network size may not necessarily improve its prediction performance. Moreover, having large network size needs more memory and computational time.

References

- [1] Jason Brownlee. *8 Tactics to Combat Imbalanced Classes in Your Machine Learning Dataset*. <http://machinelearningmastery.com/tactics-to-combat-imbalanced-classes-in-your-machine-learning-dataset/>
- [2] TryoLabs. *Why accuracy alone is a bad measure for classification tasks, and what we can do about it*. <https://tryolabs.com/blog/2013/03/25/why-accuracy-alone-bad-measure-classification-tasks-and-what-we-can-do-about-it/>
- [3] N. V. Chawla, et. al. *SMOTE: Synthetic Minority Over-sampling Technique*. <http://www.jair.org/papers/paper953.html>
- [4] *Which classifiers work well with unbalanced data?* <http://stats.stackexchange.com/questions/109177/which-classifiers-work-well-with-unbalanced-data?rq=1>
- [5] Theodoridis, Koutroumbas. *Pattern Recognition*, 4th ed. 2008
- [6] The Scipy community. *NumPy for MATLAB users*. <http://docs.scipy.org/doc/numpy-dev/user/numpy-for-matlab-users.html>